■ 2.2.18 Advanced Topic: Specifying Attributes

In addition to defining explicit transformation rules, *Mathematica* allows you to specify certain *attributes*, which describe "general properties" of functions. You should be careful to set up any attributes you need for a particular function *before* you try to give transformation rules for it. *Mathematica* needs to know the attributes that a function has in order to set the transformation rules correctly.

`Attributes[`*f*`]`	give the attributes of *f*
`Attributes[`*f*`] = {`*attr*$_1$`, `*attr*$_2$`, ...}`	set the attributes of *f*
`Attributes[`*f*`] = {}`	set *f* to have no attributes
`SetAttributes[`*f*`, `*attr*`]`	add *attr* to the attributes of *f*
`ClearAttributes[`*f*`, `*attr*`]`	remove *attr* from the attributes of *f*
`Orderless`	commutative function (arguments are sorted into standard order, and *f*`[a, b]` is equivalent to *f*`[b, a]` etc. in pattern matching)
`Flat`	associative function (arguments are "flattened out", and *f*`[`*f*`[a, b], c]` is equivalent to *f*`[a, b, c]` etc.)
`OneIdentity`	*f*`[`*f*`[`*a*`]]` etc. are equivalent to *a* for pattern matching
`Listable`	*f* is automatically "threaded" over lists that appear as arguments (e.g. *f*`[{a,`*b*`}]` becomes `{`*f*`[a], `*f*`[`*b*`]}`)
`Constant`	all derivatives of *f* are zero
`Protected`	values of *f* cannot be changed
`Locked`	attributes of *f* cannot be changed
`ReadProtected`	values of *f* cannot be read
`HoldFirst`	the first argument of *f* is not evaluated
`HoldRest`	all but the first argument of *f* is not evaluated
`HoldAll`	all the arguments of *f* are not evaluated

The complete list of attributes for symbols in *Mathematica*.

Many built-in functions have attributes; you should not try to modify them.

```
In[1]:= Attributes[Plus]
Out[1]= {Flat, Listable, OneIdentity, Orderless,
     Protected}
```

This defines a new function f to be "orderless" or commutative.

```
In[2]:= SetAttributes[f, Orderless]
```

Now the arguments of f are automatically sorted into standard order.

```
In[3]:= f[b, a, c, a]
Out[3]= f[a, a, b, c]
```

This defines g to be an associative and commutative function.

```
In[4]:= SetAttributes[g, {Flat, Orderless}]
```

The nested g's are flattened out, and the arguments are sorted.

```
In[5]:= g[g[b, a], c, g[a]]
Out[5]= g[a, a, b, c]
```

This defines a transformation rule for g.

```
In[6]:= g[x_, x_] := dg[x]
```

The attributes of g are used in applying the transformation rule.

```
In[7]:= g[a,b,a,c,d]
Out[7]= g[b, c, d, dg[g[a]]]
```

You can get rid of the g nested inside the dg by setting g to have the attribute `OneIdentity`.

```
In[8]:= SetAttributes[g, OneIdentity]
```

After you change its attributes, you should clear any values you defined for g.

```
In[9]:= Clear[g]
```

Now you must redo the definitions.

```
In[10]:= g[x_, x_] := dg[x]
```

With the attribute `OneIdentity` set, the nested g is gone.

```
In[11]:= g[a,b,a,c,d]
Out[11]= g[b, c, d, dg[a]]
```

This shows the definitions for g, including `Attributes`.

```
In[12]:= ?g

g
Attributes[g] = {Flat, OneIdentity, Orderless}
g/: g[x_, x_] := dg[x]
```

This defines h to be `Listable`.

```
In[12]:= SetAttributes[h, Listable]
```

Now h is automatically "threaded" over lists.

```
In[13]:= h[{a, b}, {c, d}, e]
Out[13]= {h[a, c, e], h[b, d, e]}
```

This defines c to be a constant in differentiation.

```
In[14]:= SetAttributes[c, Constant]
```

c is now treated as a constant.

```
In[15]:= Dt[ x^2 c[1] + x c[2] + b[0], x]
Out[15]= 2 x c[1] + c[2]
```

This stops values of i from being set or changed. You can also set the Protected attribute for a symbol simply by calling Protect[s].

```
In[16]:= SetAttributes[i, Protected]
```

Now you cannot set values of i. Unless you set the Locked attribute, you can always go back and remove the Protected attribute.

```
In[17]:= i = 7
Set::write:
    Attempt to assign to write-protected symbol i.
Out[17]= 7
```

This makes the first argument of j not be evaluated.

```
In[18]:= SetAttributes[j, HoldFirst]
```

The first argument of j is not evaluated.

```
In[19]:= j[1 + 1, 1 + 1]
Out[19]= j[1 + 1, 2]
```

This function resets the value of its argument to special. You need to set the HoldFirst attribute of j to make sure its argument is not evaluated before being used in the assignment.

```
In[20]:= j[v_] := v = special
```